13. Shell scripting

- **Shell scripts** allow several commands that would be **entered manually at a command-line interface** to be **executed automatically**, and without having to wait for a user to trigger each stage of the sequence.
- components
    - all of Linux commands
    - variables
    - looping
    - any 3rd-party or user developed programs that can be executed on Linux, like python, R, java
- from **https://en.wikipedia.org/wiki/Shell_script**

The first shell script

1. Use `touch` and `vi` to create and edit a new file
2. Type Shebeng `#!/bin/bash`
3. Type something about you
4. Type the `read` command to take the input from the keyborard and assign to the INPUT variable
5. **Please download all of scripts from https://drive.google.com/drive/folders/1sD1llq5nsidFSriCjlJJeuPR8kzYvfNG?usp=drive_link**

```
In [ ]: #!/bin/bash

# Author : Zac Gong
# Affiliation: CGU
# The first shell script follows here:

echo "What is your name?"
read INPUT
echo "Hello, $INPUT. Good to see you."
```

```
In [ ]: # Type the file name
        ! ./input_name.sh
# or bash file_name
! bash ./input_name.sh
```

Shell Basic Operators

- **Arithmetic Operators** (Kahoot)

Operator	Description	Example
+ (Addition)	Adds values on either side of the operator	`expr $a + $b` will give 30
- (Subtraction)	Subtracts right hand operand from left hand operand	`expr $a - $b` will give -10
* (Multiplication)	Multiplies values on either side of the operator	`expr $a \* $b` will give 200
/ (Division)	Divides left hand operand by right hand operand	`expr $b / $a` will give 2
% (Modulus)	Divides left hand operand by right hand operand and returns remainder	`expr $b % $a` will give 0
= (Assignment)	Assigns right operand in left operand	a = $b would assign value of b into a
== (Equality)	Compares two numbers, if both are same then returns true.	[$a == $b] would return false.
!= (Not Equality)	Compares two numbers, if both are different then returns true.	[$a != $b] would return true.

(**https://drive.google.com/uc?export=view&id=1Com2VKv8SoMYyf0bmRp7Wpp8OsiqyTe_**)

- All the following tables from **https://www.tutorialspoint.com/unix/unix-basic-operators.htm**

```
In [ ]: #!/bin/bash

val=`expr 1 + 1`
echo "Sum : $val"

## The above script will generate the following result
## Sum : 2
```

```
In [ ]: ! bash ./arith_opt.sh
```

Relational Operators

- Assume variable a holds 10 and variable b holds 20

Operator	Description	Example
-eq	Checks if the value of two operands are equal or not; if yes, then the condition becomes true.	[$a -eq $b] is not true.
-ne	Checks if the value of two operands are equal or not; if values are not equal, then the condition becomes true.	[$a -ne $b] is true.
-gt	Checks if the value of left operand is greater than the value of right operand; if yes, then the condition becomes true.	[$a -gt $b] is not true.
-lt	Checks if the value of left operand is less than the value of right operand; if yes, then the condition becomes true.	[$a -lt $b] is true.
-ge	Checks if the value of left operand is greater than or equal to the value of right operand; if yes, then the condition becomes true.	[$a -ge $b] is not true.
-le	Checks if the value of left operand is less than or equal to the value of right operand; if yes, then the condition becomes true.	[$a -le $b] is true.

(https://drive.google.com/uc?export=view&id=1Cns9fo-xTipR7Mf7GKpszmqX1Z8vFFc9)

String Operators

- Assume variable a holds 10 and variable b holds 20

Operator	Description	Example
=	Checks if the value of two operands are equal or not; if yes, then the condition becomes true.	[$a = $b] is not true.
!=	Checks if the value of two operands are equal or not; if values are not equal then the condition becomes true.	[$a != $b] is true.
-z	Checks if the given string operand size is zero; if it is zero length, then it returns true.	[-z $a] is not true.
-n	Checks if the given string operand size is non-zero; if it is nonzero length, then it returns true.	[-n $a] is not false.
str	Checks if **str** is not the empty string; if it is empty, then it returns false.	[$a] is not false.

(https://drive.google.com/uc?export=view&id=1Cnw5iac4DHpxRBwMC2k0qRjDYe9UPTLp)

Boolean Operators

- Assume variable a holds 10 and variable b holds 20

Operator	Description	Example
!	This is logical negation. This inverts a true condition into false and vice versa.	[! false] is true.
-o	This is logical **OR**. If one of the operands is true, then the condition becomes true.	[$a -lt 20 -o $b -gt 100] is true.
-a	This is logical **AND**. If both the operands are true, then the condition becomes true otherwise false.	[$a -lt 20 -a $b -gt 100] is false.

(**https://drive.google.com/uc?export=view&id=1CoC1OZKhgZ0BTA492LeEFmu8z1LV_VI3**)

File Test Operators

- Assume variable a holds 10 and variable b holds 20

Operator	Description	Example
-b file	Checks if file is a block special file; if yes, then the condition becomes true.	[-b $file] is false.
-c file	Checks if file is a character special file; if yes, then the condition becomes true.	[-c $file] is false.
-d file	Checks if file is a directory; if yes, then the condition becomes true.	[-d $file] is not true.
-f file	Checks if file is an ordinary file as opposed to a directory or special file; if yes, then the condition becomes true.	[-f $file] is true.
-g file	Checks if file has its set group ID (SGID) bit set; if yes, then the condition becomes true.	[-g $file] is false.
-k file	Checks if file has its sticky bit set; if yes, then the condition becomes true.	[-k $file] is false.
-p file	Checks if file is a named pipe; if yes, then the condition becomes true.	[-p $file] is false.
-t file	Checks if file descriptor is open and associated with a terminal; if yes, then the condition becomes true.	[-t $file] is false.
-u file	Checks if file has its Set User ID (SUID) bit set; if yes, then the condition becomes true.	[-u $file] is false.
-r file	Checks if file is readable; if yes, then the condition becomes true.	[-r $file] is true.
-w file	Checks if file is writable; if yes, then the condition becomes true.	[-w $file] is true.
-x file	Checks if file is executable; if yes, then the condition becomes true.	[-x $file] is true.
-s file	Checks if file has size greater than 0; if yes, then condition becomes true.	[-s $file] is true.
-e file	Checks if file exists; is true even if file is a directory but exists.	[-e $file] is true.

(**https://drive.google.com/uc?export=view&id=1Cnua-EdDnQFd5w2UiXABTbst3pCnv3Du**)

Shell Decision Making

- if...else is a commonly-used, decision-making statement.
    - if...fi statement
    - if...else...fi statement
    - if...elif...else...fi statement

In [ ]:
```
#!/bin/bash

# Author : Zac Gong
# Affiliation: CGU
# The first shell script follows here:

echo "What is your name?"
read INPUT

if [ "$INPUT" == ""]; then
    echo "Null input. Program stopped."
    exit
else
    echo "Hello, $INPUT. Good to see you."
fi
```

In [ ]:
```
! bash ./input_name_decision_making.sh
```

use getopts to parse command-line arguments and display your input and output file names

- getopts command: retrieve options and their arguments
- for example, two required arguments as input and output file names

In [ ]:
```
! bash ./print_io.sh -i input.fas -o output.fas
```

In [ ]:
```
#!/bin/bash
# Retrieve file names and print out

### get args
while getopts i:o: option
do
case "${option}"
in
i) i=${OPTARG};;
o) o=${OPTARG};;
esac
done

### check args
if [ "$i" == "" -o "$o" == "" ]
then
        echo "Input Error, please make sure"
        echo "-i as input file name"
        echo "-o as output file name"
        echo "Usage: sh printInputOutput.sh -i input.fas -o output.fas"
        exit
fi

echo "input file name: "$i
echo "output file name: "$o
```

Read a fasta file and generate a CSV file

In [ ]:
```
! bash ./generate_csv.sh -i input.fas -o output.fas
```

In [ ]:
```
#!/bin/bash

### get args
while getopts i:o: option
do
case "${option}"
in
i) i=${OPTARG};;
o) o=${OPTARG};;
esac
done

### check args
if [ "$i" == "" -o "$o" == "" ]
then
    echo "Input Error, please make sure"
    echo "-i as input file name"
    echo "-o as output file name"
    echo "Usage: bash generate_csv.sh -i input.fas -o output.fas"
    exit
fi
echo "input file name: "$i
echo "output file name: "$o

# step 1. create CSV file
echo "virus,location,strain,year,accession_number,collection_date,area" > $o
# step 2. retrieve meta data and stream editor
grep ">" $i | sed 's/>//g' | sed 's/\// | /g' | sed 's/ | /,/g' >> $o
```

In [3]:
```
# execute this shell script
! bash ./generate_csv.sh -i demo.fas -o metadata.csv
```

```
input file name: demo.fas
output file name: metadata.csv
```

In [4]:
```
! head -3 metadata.csv
```

```
virus,location,strain,year,accession_number,collection_date,area
hCoV-19,USA,CA-CZB-2021,2020,EPI_ISL_486273,2020-05-04,NorthAmerica
hCoV-19,Wuhan,WIV04,2019,EPI_ISL_402124,2019-12-30,China
```